Báo Cáo Dự Án
RetinaNet + ConvNeXt cho Bài Toán Phát Hiện Hộp Thuốc

Dự án: `retinanet_model`

March 6, 2026

Contents

1 Giới thiệu (Introduction) 2
- 1.1 Tổng quan các phương pháp object detection . . . 2
- 1.2 Related Work . . . 2
- 1.3 Lý do chọn RetinaNet . . . 2

2 Tiền xử lý dữ liệu (Data Preprocessing) 3
- 2.1 Thống kê dataset và phân bố lớp . . . 4
- 2.2 Bounding box statistics . . . 4

3 Phương pháp (Methodology) 5
- 3.1 RetinaNet . . . 5
- 3.2 Backbone ConvNeXt . . . 5
- 3.3 Feature Pyramid Network (FPN) . . . 6
- 3.4 Detection Head . . . 6
- 3.5 Anchor Boxes . . . 6
- 3.6 Loss Function . . . 6

4 Cài đặt hệ thống (Implementation) 7
- 4.1 Thiết lập huấn luyện . . . 7
- 4.2 Quy trình thực thi . . . 8

5 Đánh giá và kết quả (Evaluation and Results) 8
- 5.1 COCO Metrics . . . 8
- 5.2 Detection Metrics . . . 8
- 5.3 Kết quả định lượng . . . 8
- 5.4 Phân tích confusion matrix . . . 9

6 Thảo luận và hướng phát triển (Discussion and Future Work) 10

1 Giới thiệu (Introduction)

Bài toán phát hiện đối tượng (object detection) đóng vai trò quan trọng trong nhiều ứng dụng thị giác máy tính, đặc biệt trong các hệ thống quản lý kho dược phẩm, tự động hóa trong nhà thuốc và các hệ thống kiểm kê thuốc. Trong các kịch bản này, khả năng phát hiện và phân loại chính xác các hộp thuốc giúp giảm sai sót của con người và nâng cao hiệu quả vận hành.

Tuy nhiên, việc phát hiện các hộp thuốc cũng gặp nhiều thách thức. Nhiều loại thuốc có hình dạng và kích thước tương tự nhau, bao bì có màu sắc gần giống nhau, và các hộp thuốc thường xuất hiện với nhiều góc nhìn khác nhau trong ảnh. Điều này khiến việc phân biệt giữa các lớp thuốc trở nên khó khăn hơn so với các bài toán phát hiện đối tượng thông thường.

1.1 Tổng quan các phương pháp object detection

Các mô hình phát hiện đối tượng hiện đại thường được chia thành hai hướng chính:

- **Two-stage detector**: đại diện là Faster R-CNN. Mô hình tạo proposal ở stage 1 và phân loại/hồi quy ở stage 2, thường cho độ chính xác cao nhưng tốc độ suy luận thấp hơn.
- **One-stage detector**: đại diện là YOLO, SSD, RetinaNet. Các mô hình này dự đoán trực tiếp lớp và bounding box từ feature map nên có tốc độ nhanh, phù hợp bài toán thời gian thực.

1.2 Related Work

Faster R-CNN là mô hình two-stage kinh điển, mạnh về độ chính xác nhưng có độ trễ lớn khi triển khai thực tế. SSD và các dòng YOLO cải thiện đáng kể tốc độ suy luận nhờ kiến trúc one-stage. Tuy nhiên, one-stage detector thường gặp vấn đề mất cân bằng nghiêm trọng giữa foreground và background. RetinaNet khắc phục điểm yếu này bằng Focal Loss, giúp giảm trọng số của các mẫu dễ và tập trung tối ưu vào mẫu khó.

1.3 Lý do chọn RetinaNet

Trong bối cảnh bài toán hộp thuốc đa lớp, hệ thống cần đồng thời đạt độ chính xác cao và thời gian suy luận hợp lý. RetinaNet là lựa chọn phù hợp vì:

- Kế thừa lợi thế tốc độ của one-stage detector.
- Ổn định hơn trước class imbalance nhờ Focal Loss.
- Tương thích tốt với backbone mạnh (ConvNeXt) và học đa tỉ lệ qua FPN.

Trong dự án này, chúng tôi xây dựng một hệ thống phát hiện đối tượng để nhận diện 12 loại hộp thuốc khác nhau. Mô hình được xây dựng dựa trên kiến trúc RetinaNet với backbone ConvNeXt-Tiny kết hợp với Feature Pyramid Network (FPN). Hệ thống hỗ trợ nhiều định dạng annotation khác nhau (bounding box hoặc polygon), đồng thời cung cấp một pipeline hoàn chỉnh từ tiền xử lý dữ liệu, huấn luyện mô hình, đến đánh giá kết quả.

Các đóng góp chính của nghiên cứu bao gồm:

1. Đề xuất pipeline RetinaNet + ConvNeXt-Tiny cho bài toán phát hiện 12 loại hộp thuốc.
2. Thiết kế bộ nạp dữ liệu linh hoạt cho cả YOLO bbox và polygon annotation.

3. Xây dựng quy trình thực nghiệm có thể tái lập từ huấn luyện, chọn checkpoint đến đánh giá.

4. Đánh giá toàn diện bằng COCO metrics, detection metrics và phân tích lỗi bằng confusion matrix/visualization.

2 Tiền xử lý dữ liệu (Data Preprocessing)

Bộ dữ liệu sử dụng trong dự án gồm các hình ảnh chứa nhiều loại hộp thuốc cùng với các annotation tương ứng. Các annotation được lưu theo định dạng YOLO, trong đó mỗi ảnh có một file nhãn tương ứng.

Hai định dạng annotation được hỗ trợ gồm:

1. **YOLO bounding box**: `class x_center y_center width height`. Các giá trị tọa độ thường được chuẩn hóa trong khoảng $[0, 1]$.

2. **Polygon annotation**: `class x1 y1 x2 y2 ... xn yn`.

Trong quá trình tiền xử lý, các polygon sẽ được chuyển đổi sang bounding box bằng cách tính:

$$x_{\text{min}} = \min(x_i), \quad x_{\text{max}} = \max(x_i), \quad y_{\text{min}} = \min(y_i), \quad y_{\text{max}} = \max(y_i)$$

sau đó tạo bounding box bao quanh polygon.

Ngoài ra, các bước xử lý dữ liệu khác cũng được áp dụng:

- Chuyển đổi tọa độ chuẩn hóa sang tọa độ pixel.
- Kiểm tra và loại bỏ các bounding box không hợp lệ.
- Đảm bảo nhãn lớp nằm trong khoảng hợp lệ.

Bộ dữ liệu được chia thành ba tập:

- Training set.
- Validation set.
- Test set.

Để cải thiện khả năng tổng quát hóa của mô hình, một số kỹ thuật augmentation được áp dụng:

- Resize ảnh về kích thước 640×640.
- Random horizontal flip với xác suất 0.5.
- Chuẩn hóa dữ liệu theo ImageNet mean và standard deviation.

Những bước tiền xử lý này giúp đảm bảo dữ liệu đầu vào đồng nhất và tăng độ robust của mô hình.

2.1 Thống kê dataset và phân bố lớp

Thống kê trực tiếp từ toàn bộ tập dữ liệu cho thấy:

- Tổng số ảnh: 14932 (train: 10443, valid: 2233, test: 2256).
- Tổng số bounding box hợp lệ: 20097.
- Số lớp: 12.

Phân bố lớp có mất cân bằng ở mức vừa phải, với lớp nhiều nhất là `class_6` (2281 mẫu) và lớp ít nhất là `class_11` (1327 mẫu), tỉ lệ mất cân bằng max / min ≈ 1.72.

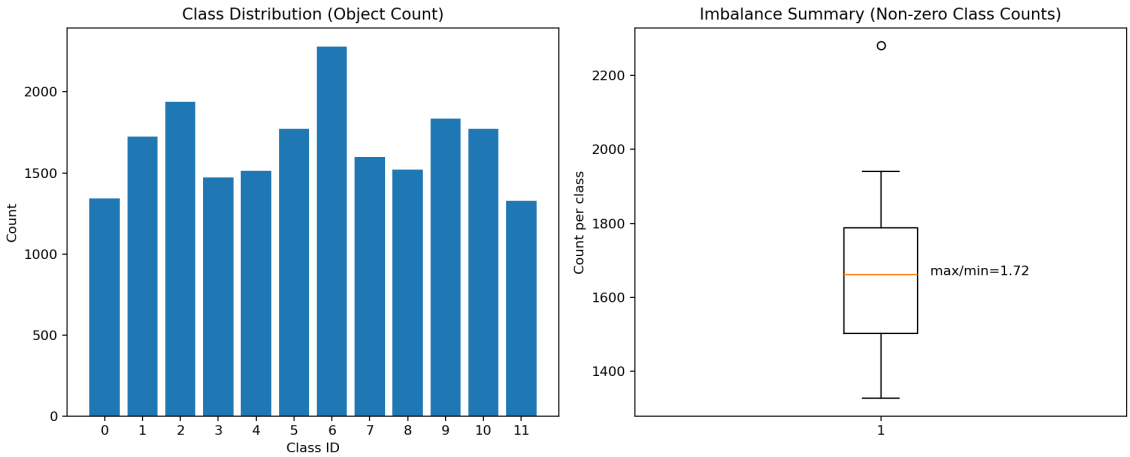

Figure 1: Phân bố số lượng đối tượng theo lớp và tóm tắt mức độ mất cân bằng

2.2 Bounding box statistics

Thống kê hình học bounding box (trên tọa độ chuẩn hóa) cho thấy kích thước đối tượng phân tán đáng kể, phù hợp với việc sử dụng FPN đa tỉ lệ:

Đại lượng	**Mean**	**Median (P50)**
Width	0.5858	0.5938
Height	0.5254	0.4938
Area	0.3239	0.2903
Aspect ratio (w/h)	1.3826	-

Table 1: Thống kê hình học bounding box trên toàn bộ dataset

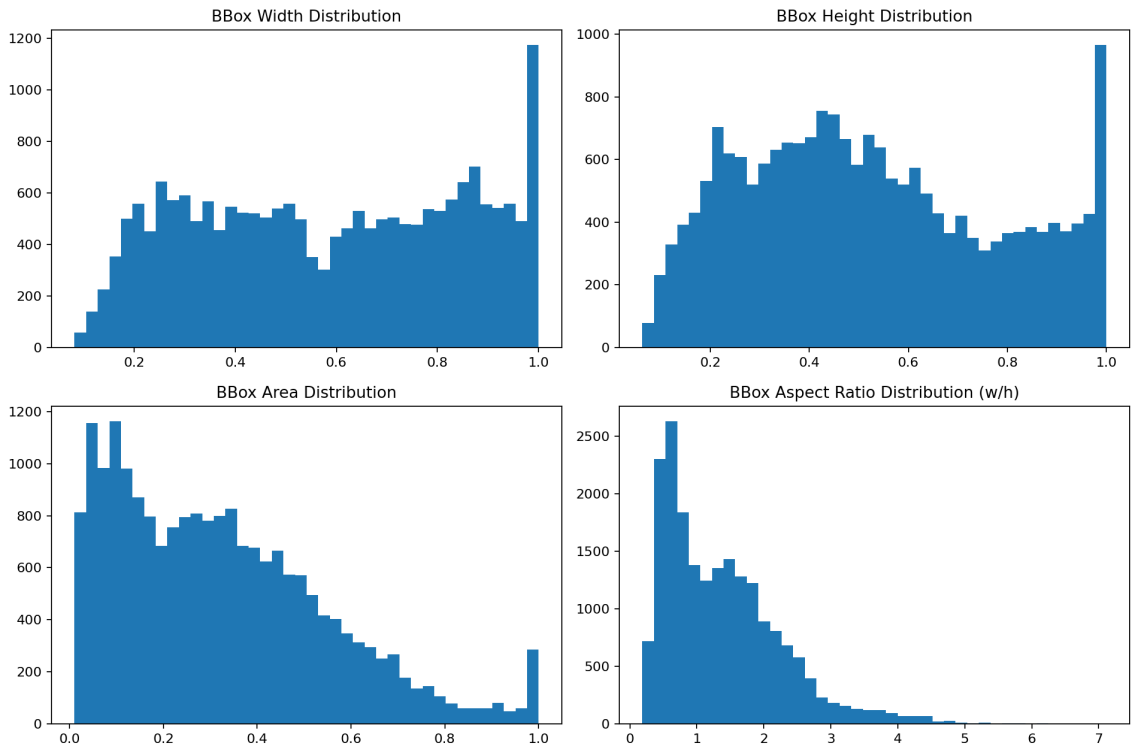

Figure 2: Phân bố width, height, area và aspect ratio của bounding box

3 Phương pháp (Methodology)

Hệ thống phát hiện đối tượng được xây dựng dựa trên kiến trúc RetinaNet với backbone ConvNeXt và Feature Pyramid Network (FPN).

3.1 RetinaNet

RetinaNet là một mô hình phát hiện đối tượng một bước, dự đoán trực tiếp bounding box và xác suất lớp từ feature map.

Điểm nổi bật của RetinaNet là Focal Loss, giúp giảm ảnh hưởng của các mẫu dễ (background) và tập trung vào các mẫu khó trong quá trình huấn luyện. Điều này đặc biệt quan trọng trong bài toán object detection, nơi số lượng background thường lớn hơn rất nhiều so với số lượng đối tượng.

Về mặt toán học, với logits z và xác suất $p = \sigma(z)$, focal loss cho nhị phân được viết là:

$$\mathcal{L}_{\text{focal}}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

trong đó $p_t = p$ nếu mẫu là dương và $p_t = 1 - p$ nếu mẫu là âm. Thành phần $(1 - p_t)^\gamma$ làm giảm đóng góp gradient từ các mẫu dễ, giúp tối ưu tập trung vào các mẫu khó.

3.2 Backbone ConvNeXt

ConvNeXt-Tiny được sử dụng làm backbone để trích xuất đặc trưng từ ảnh đầu vào. ConvNeXt là một kiến trúc CNN hiện đại được thiết kế dựa trên nhiều nguyên lý từ Vision Transformer nhưng vẫn giữ ưu điểm về hiệu quả tính toán của convolution.

Backbone này tạo ra các feature map đa tầng: C_2, C_3, C_4, C_5. Các tầng này biểu diễn thông tin hình ảnh ở nhiều mức độ chi tiết khác nhau.

3.3 Feature Pyramid Network (FPN)

Để phát hiện đối tượng ở nhiều kích thước khác nhau, Feature Pyramid Network được sử dụng để kết hợp thông tin từ nhiều tầng feature map.

FPN thực hiện quá trình:

- Top-down feature fusion.
- Lateral connections.

Kết quả là tạo ra các feature pyramid: P_3, P_4, P_5, P_6, P_7. Các feature này giúp mô hình phát hiện tốt cả đối tượng nhỏ và lớn.

3.4 Detection Head

Ở mỗi mức của feature pyramid, RetinaNet sử dụng hai mạng con:

- **Classification head**: dự đoán xác suất lớp của đối tượng.
- **Regression head**: dự đoán offset của bounding box so với anchor box.

3.5 Anchor Boxes

Anchor boxes được đặt tại mỗi vị trí của feature map với cấu hình:

- Anchor sizes: 32, 64, 128, 256, 512.
- Aspect ratios: 0.5, 1.0, 2.0.
- Scales: $1, 2^{1/3}, 2^{2/3}$.

Như vậy, mỗi vị trí tạo ra 9 anchor boxes.

Với anchor $a = (a_x, a_y, a_w, a_h)$ và ground-truth $g = (g_x, g_y, g_w, g_h)$, biến đổi encode được dùng trong huấn luyện:

$$t_x = \frac{g_x - a_x}{a_w}, \quad t_y = \frac{g_y - a_y}{a_h}, \quad t_w = \log \frac{g_w}{a_w}, \quad t_h = \log \frac{g_h}{a_h}$$

Khi suy luận, mô hình dự đoán $\hat{t} = (\hat{t}_x, \hat{t}_y, \hat{t}_w, \hat{t}_h)$ và decode ngược lại thành box dự đoán:

$$\hat{g}_x = \hat{t}_x a_w + a_x, \quad \hat{g}_y = \hat{t}_y a_h + a_y, \quad \hat{g}_w = e^{\hat{t}_w} a_w, \quad \hat{g}_h = e^{\hat{t}_h} a_h$$

3.6 Loss Function

Mô hình được huấn luyện với hai thành phần loss:

- Classification loss: Focal Loss.
- Regression loss: Smooth L1 Loss.

Smooth L1 cho sai lệch d được định nghĩa:

$$\mathcal{L}_{\text{smoothL1}}(d) = \begin{cases} \frac{1}{2}\frac{d^2}{\beta}, & |d| < \beta \\ |d| - \frac{1}{2}\beta, & \text{ngược lại} \end{cases}$$

Hàm mục tiêu tổng quát của mạng:

$$\mathcal{L} = \lambda_{cls}\mathcal{L}_{focal} + \lambda_{box}\mathcal{L}_{smoothL1}$$

Trong triển khai hiện tại, hệ số mặc định được đặt cân bằng ($\lambda_{cls} = \lambda_{box} = 1$).

Anchor được gán nhãn dựa trên IoU:

- Positive: IoU ≥ 0.5.
- Negative: IoU < 0.4.
- Ignore: $0.4 \leq$ IoU < 0.5.

Với cách gán nhãn này, mô hình cân bằng tốt giữa độ nhạy phát hiện và khả năng loại bỏ nhiễu nền, đặc biệt khi số lượng background lớn hơn đối tượng.

Sau bước dự đoán, NMS được áp dụng để loại bỏ box trùng lặp theo ngưỡng IoU, giúp tăng độ chính xác thực tế của kết quả đầu ra.

4 Cài đặt hệ thống (Implementation)

Hệ thống được cài đặt bằng Python và PyTorch, theo quy trình thực nghiệm có thể tái lập gồm ba giai đoạn: (i) khởi tạo dữ liệu và augmentation, (ii) huấn luyện tối ưu tham số, (iii) đánh giá định lượng và định tính.

4.1 Thiết lập huấn luyện

Ảnh đầu vào được chuẩn hóa về kích thước 640×640. Mô hình được huấn luyện với AdamW, mixed precision (AMP), gradient clipping và checkpointing theo chất lượng validation.

Các tham số huấn luyện chính:

Tham số	**Giá trị**
Image size	640
Batch size	16
Epochs	30
Learning rate	1×10^{-4}
Weight decay	0.05
Number of classes	12

Trong quá trình huấn luyện, hệ thống lưu checkpoint tốt nhất theo tiêu chí AP@0.50:0.95 và checkpoint cuối cùng để hỗ trợ phân tích hội tụ cũng như phục hồi phiên làm việc.

4.2 Quy trình thực thi

Quy trình thực nghiệm được thực hiện theo thứ tự:

- Huấn luyện mô hình trên tập train và theo dõi metric trên validation.
- Chọn mô hình tốt nhất theo AP@0.50:0.95.
- Đánh giá trên tập test bằng cả COCO metrics và detection metrics.
- Phân tích lỗi thông qua confusion matrix và trực quan hóa dự đoán.

Lệnh chạy thực nghiệm:

```
pip install -r requirements.txt
python -m train
python -m test
python -m test_confusion
python -m test_visualize_gt_pred
```

5 Đánh giá và kết quả (Evaluation and Results)

Mô hình được đánh giá bằng hai nhóm chỉ số:

5.1 COCO Metrics

- AP@0.50:0.95: average precision ở nhiều ngưỡng IoU.
- AP@0.50.
- AP@0.75.
- Average Recall (AR).

5.2 Detection Metrics

Ngoài ra còn sử dụng các chỉ số:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}, \quad \text{Detection Accuracy} = \frac{TP}{TP + FP + FN}$$

5.3 Kết quả định lượng

Dựa trên đồ thị huấn luyện và ma trận nhầm lẫn thu được, hiệu năng mô hình ở cuối quá trình huấn luyện có thể tóm tắt như sau:

Chỉ số	Giá trị xấp xỉ
Train Loss (cuối)	≈ 0.01
Validation Loss (cuối)	≈ 0.015
AP@0.50:0.95	≈ 0.87
AP@0.50	≈ 0.99
AP@0.75	≈ 0.98
AR@0.50:0.95	≈ 0.90

Table 2: Tổng hợp kết quả chính từ quá trình huấn luyện và đánh giá

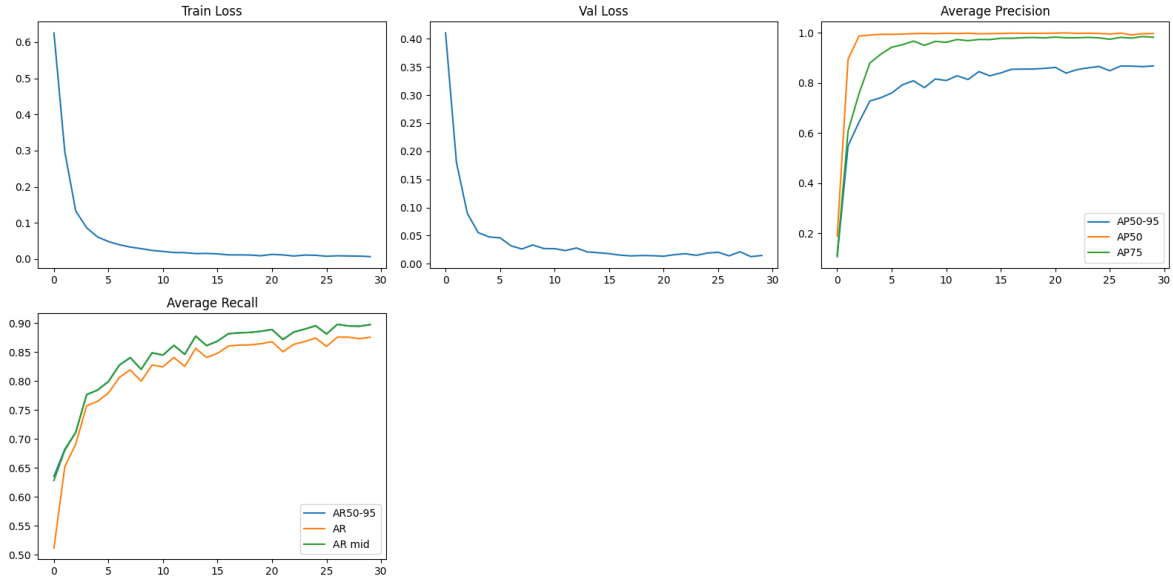

Figure 3: Diễn biến loss, AP và AR theo epoch

5.4 Phân tích confusion matrix

Ma trận nhầm lẫn cho thấy đa số mẫu nằm trên đường chéo chính, phản ánh khả năng phân loại tốt giữa các lớp thuốc. Tổng số dự đoán đúng trên 12 lớp đạt 2894 mẫu đúng trong tổng 2959 ground-truth đối tượng (xấp xỉ recall vi mô $\approx 97.8\%$).

Một số điểm cần lưu ý:

- Nhầm lẫn lớn nhất xuất hiện ở các cặp lớp có đặc trưng thị giác gần nhau, đặc biệt liên quan đến `class_6` và các lớp lân cận.
- Số lượng false positive từ nền vào các lớp `class_6`, `class_9`, `class_10` còn tương đối cao.
- Một số false negative vẫn xuất hiện ở các lớp có kích thước nhỏ hoặc góc chụp khó.

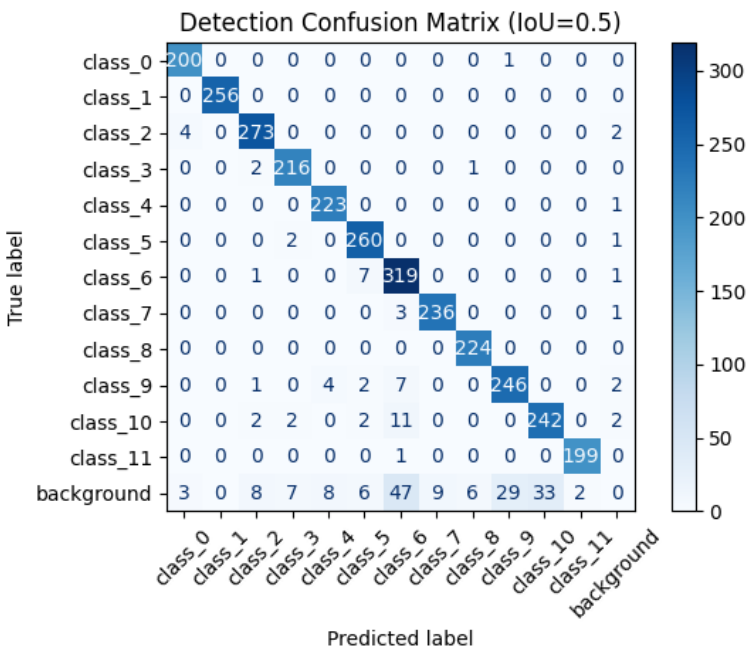

Figure 4: Ma trận nhầm lẫn trên tập test (IoU=0.5)

Các công cụ trực quan hóa được sử dụng gồm:

- Confusion matrix.
- So sánh bounding box dự đoán và ground truth.
- Visualization trên ảnh test.

Những công cụ này giúp phân tích các lỗi phổ biến của mô hình.

6 Thảo luận và hướng phát triển (Discussion and Future Work)

Kết quả thực nghiệm cho thấy mô hình hội tụ ổn định và đạt độ chính xác cao trên tập dữ liệu hiện tại. Việc kết hợp ConvNeXt + FPN + RetinaNet phát huy hiệu quả tốt trong bối cảnh dữ liệu đa lớp và có sự mất cân bằng giữa foreground/background.

Mặc dù vậy, vẫn còn một số hạn chế cần cải thiện.

Thứ nhất, hiệu năng mô hình phụ thuộc nhiều vào chất lượng và sự đa dạng của dữ liệu huấn luyện. Một số loại thuốc có bao bì rất giống nhau, dẫn đến khả năng nhầm lẫn giữa các lớp.

Thứ hai, cấu hình anchor hiện tại có thể chưa tối ưu với phân bố bounding box trong dataset. Trong tương lai có thể áp dụng K-means clustering để tìm anchor tối ưu.

Ngoài ra, một số hướng cải thiện khác gồm:

- Phân tích hiệu năng theo từng lớp thuốc.
- Vẽ precision–recall curve cho từng lớp.
- Sử dụng EMA (Exponential Moving Average) để ổn định huấn luyện.
- Tích hợp TensorBoard hoặc Weights & Biases để theo dõi thí nghiệm.

- Đánh giá tốc độ suy luận (FPS) để phục vụ triển khai thực tế.

Những cải tiến này có thể giúp hệ thống phát hiện thuốc trở nên chính xác và ổn định hơn trong các ứng dụng thực tế.